

Hermes 的记忆系统

Rainbow Team

2026-09-22

目录

1 引言	1
1.1 设计目标	1
1.2 整体分层	1
2 SessionDB: 本地 SQLite	2
2.1 物理结构	2
2.2 写入路径单入口	3
2.3 FTS5: 让历史可搜	4
2.4 Tokenizer 的选择与中文增强	4
2.5 查询语法	5
2.6 语音场景中的 FTS5	6
2.6.1 <code>recall_about</code> 使用	6
3 Prompt Cache 与上下文压缩	7
3.1 大模型读写计费	7
3.2 Cache 约束	7
3.3 上下文压缩	8
4 记忆能力扩展	9
4.1 内置本地记忆: MEMORY.md + USER.md	9
4.2 多说话人本地记忆	9
4.2.1 利用 LLM 纪律性分辨用户	10
4.2.2 Memory 预过滤	10
4.3 MemoryProvider 抽象	10
4.4 内置可选 backend	11
5 插件系统	11
5.1 四条发现路径	11
5.2 插件依赖拓扑	12
6 总结	13
7 附录 A: 对照参考 — Claude Code 的记忆与压缩	14

1 引言

LLM 的请求-响应是无状态的——模型每一次只能看到当前 prompt 中的内容，对上一轮、上一天发生的事一无所知。最朴素的对策是把整段历史重新提交到 prompt 中，但 context window 存在硬上限，长对话很快即会突破上限；即便未突破上限，token 成本也会随轮数线性增长。

记忆系统就是用来打破“prompt=一切”这个局面的：把对话历史以结构化、可搜索、可压缩、可外挂的形式持久化到 prompt 之外，需要时再按需取回。Hermes 把这件事拆成几层正交的关注点——本地存储、全文检索、上下文压缩、外挂语义记忆——默认部署只用一份 SQLite 就能跑通，更高阶的能力以插件形式叠加，核心不为它们付出复杂度。

① 关于本文出现的 \$HERMES_HOME

\$HERMES_HOME 是 hermes 的运行时目录。默认部署下它解析到 `~/.hermes/`；使用 profile 模式分实例时（`hermes -p <name>`），它解析到 `~/.hermes/profiles/<name>/`，`config.yaml` / `state.db` / `plugins/` / `vendor/` 等所有运行时状态都按 profile 隔离。

1.1 设计目标

记忆系统的设计由四条硬约束规约，均源自真实的工程现实：

目标	内容
零外部依赖	默认安装能开箱跑通，云端 backend 是可选项而不是前提
Prompt cache 友好	保持前缀一致性可以显著降低 token 用量
可扩展	跨设备 / 语义召回 / 长期摘要等更高级的能力通过插件挂入
可审计	每条消息都带时间戳、token 估算、来源平台、是否压缩等元数据

表 1.1 Hermes 记忆系统的四个设计目标

1.2 整体分层

Hermes 把记忆相关的代码切成五层。最上面是对话主循环，最下面是 SQLite 物理文件；中间三层是“插件友好”的扩展点，外部代码只要遵守接口契约就可以替换实现，不需要改 Hermes 本身。

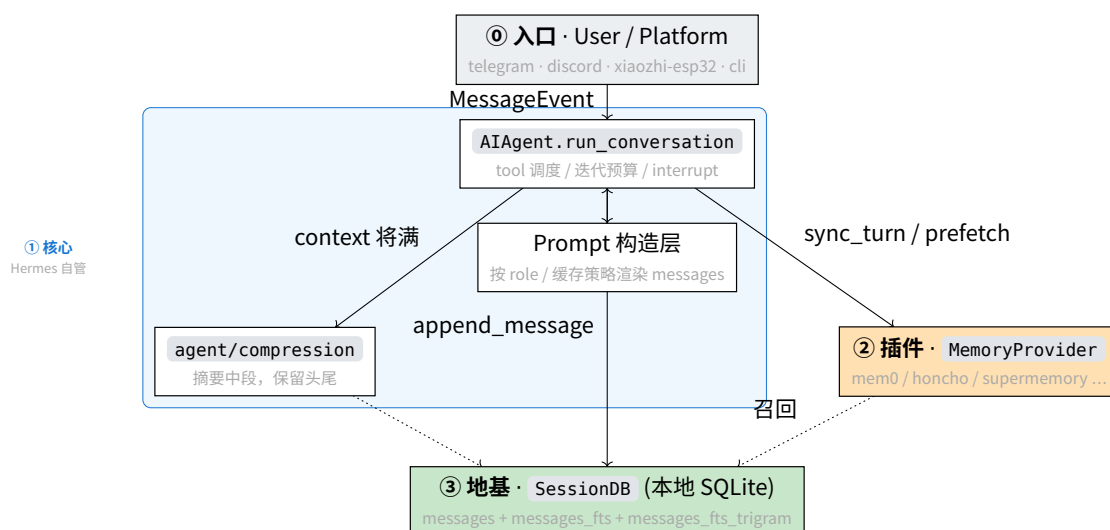


图 1.1 五层栈：① 平台入口把消息送入 ① Hermes 核心；主循环跟 Prompt 构造层来回交互驱动每一轮对话；context 接近窗口上限时分流到 `agent/compression` 做摘要；用户偏好 / 跨 session 知识则下放到 ② `MemoryProvider` 外挂 backend；所有消息最终经 `append_message` 写入 ③ `SessionDB`，`agent/compression` 和 `MemoryProvider` 也都从这里召回历史。

层级	职责
AIAgent.run_conversation	对话主循环、tool 调度、interrupt 控制、迭代预算
Prompt 构造层	把 messages 按 role / 缓存策略渲染成 API 入参
MemoryProvider 抽象	外挂语义记忆 backend，每轮 sync_turn / 启动 prefetch
agent/compression	上下文将近 context window 时，摘要中段保留头尾
SessionDB (SQLite)	本地会话与消息的物理存储 + FTS5 全文索引

表 1.2 Hermes 记忆系统的内部分层

最下面两层（SessionDB + 压缩）是 Hermes 自身掌控的“地基”，上面三层均允许通过插件机制替换。其好处是：默认部署仅依赖本地 SQLite 即可运行，需要更高阶能力时再外挂；插件本身无须 fork 主仓，只需放置于 `$HERMES_HOME/plugins/` 目录或通过 `entry_point` 安装，便会被自动发现。

2 SessionDB：本地 SQLite

✓ 为什么是 SQLite

嵌入式、零服务、单文件方便备份、自带高质量 **FTS5** 索引、支持 WAL 并发写、跨平台稳定。一个 `$HERMES_HOME/state.db` 文件即整个状态。使用自带的 FTS5 可跨 session 搜历史。

2.1 物理结构

整个 `$HERMES_HOME/state.db` 由几张协同工作的表组成——`sessions` / `messages` 是核心关系表，加上 2 份 FTS5 倒排索引、`compression_locks` 这把 per-session 独占锁，以及 `state_meta` / `schema_version` 两张辅助 KV 表。下图是 ER 视图：

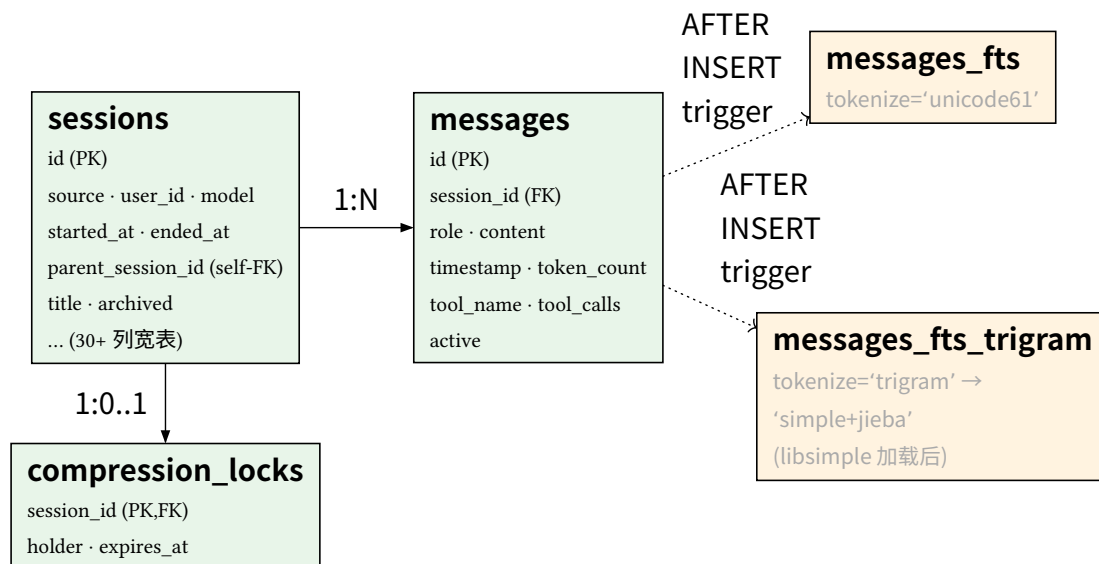


图 2.1 SessionDB 的实体关系：sessions 1-N messages (外键 `messages.session_id`)；messages 通过 `AFTER INSERT/UPDATE/DELETE` 触发器自动同步到两份 FTS5 索引；`compression_locks` 是按 session 串行化压缩的独占锁，正常会话生命周期内为空。

表	作用
<code>sessions</code>	每个对话一行（30+ 列宽表）： <code>id</code> / <code>source</code> / <code>user_id</code> / <code>model</code> / <code>started_at</code> / <code>ended_at</code> / token & cost 统计 / <code>parent_session_id</code> （自引用 FK，压缩链） / <code>title</code> / <code>archived</code> 等。
<code>messages</code>	每条消息一行： <code>role</code> / <code>content</code> / <code>timestamp</code> / <code>token_count</code> / <code>tool_name</code> / <code>tool_calls</code> / <code>active</code> （压缩软删标记）
<code>messages_fts</code>	FTS5 倒排索引（默认 <code>unicode61</code> 分词器，中英文混排都能搜）
<code>messages_fts_trigram</code>	FTS5 CJK 索引，默认 <code>trigram</code> ，加载 <code>libsimple</code> 扩展时自动切换为 <code>simple+jieba</code> （参见 小节 2.4）
<code>compression_locks</code>	按 session 的独占压缩锁（ <code>session_id</code> PK + <code>holder</code> + <code>expires_at</code> ），防止 gateway / CLI / <code>worktree agent</code> 并发压缩 race；正常态为空

表 2.1 SessionDB 的主要表与字段

数据库启用 WAL 模式（`PRAGMA journal_mode=WAL`）以允许“读不阻塞写”，再配合 `synchronous=NORMAL` 把 fsync 频率压低到每个 checkpoint 一次。这样的取舍让单机吞吐量足够支撑会话级写入（远低于真正的写密集型场景），同时避免了 每条消息都同步到磁盘的延迟。

2.2 写入路径单入口

所有对消息的修改都通过 `SessionDB.append_message()`，包括 user / assistant / tool result 三种 role。把入口收敛成一个函数有两个好处：FTS5 同步可以靠 SQLite 的 `AFTER INSERT` 触发器保证一致性；上层代码不需要关心索引、不需要关心 token 估算。

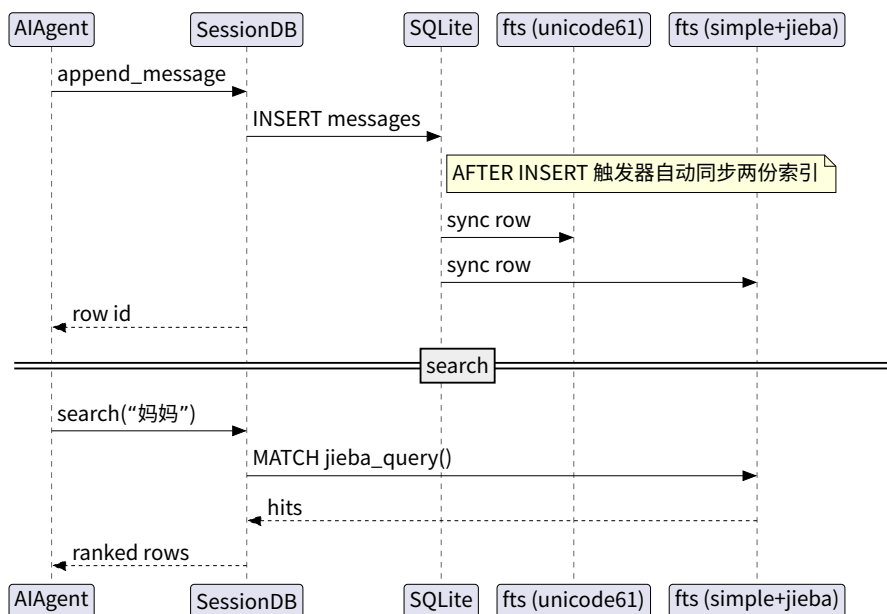


图 2.2 写路径与查询路径：单一入口 `append_message`，索引由触发器自动维护；搜索直接 `MATCH jieba_query(?)` 命中 `simple+jieba` 分词器。

写入逻辑的核心是把（`session_id`, `role`, `content`, ...`metadata`）写到 `messages` 表，`AFTER INSERT / DELETE / UPDATE` 触发器自动把消息内容复刻到 `messages_fts` 和 `messages_fts_trigram` 两份倒排索引上。索引一致性从应用层下沉到数据库层——上层只要 `INSERT INTO messages`，两份索引自动跟上。代价是每次写消息要多做两次倒排索引更新。

2.3 FTS5：让历史可搜

① 全文索引 vs 向量库

SQLite 内置 FTS5 全文索引，零额外进程；`MATCH` 语法支持布尔运算、短语、近邻、前缀；BM25 排名和 `snippet()` 高亮都开箱即用。向量库能补语义近邻召回但需要额外的进程和存储。

`messages` 上同时挂着两份 FTS5 索引，`AFTER INSERT/UPDATE/DELETE` 触发器把每条新消息复刻到两张表上。两张表的分词器不同，分别解决不同语种的检索问题：

表	分词器	目标查询
<code>messages_fts</code>	<code>unicode61</code>	英文 / ASCII。按 Unicode 词边界切分，自然对应“word”概念
<code>messages_fts_trigram</code>	<code>trigram</code> 或 <code>simple+jieba</code>	中文（CJK）。trigram 是滑窗 3-gram 兜底；扩展 jieba 分词，得到精确召回

表 2.2 FTS5 双索引：每张虚拟表对应一种分词器，按查询语种分流

备注

`unicode61` 不是“原生支持 CJK 的分词器”——它对中文的处理是按字符切分（“大别山”切成“大”“别”“山”），把英文那套“词命中”逻辑套到中文上非常低效：搜“大别山项目”会被拆成 5 个独立 token 做 AND，既出假阳性也漏掉真正的短语匹配。这正是单独挂第二张表把 CJK 索引分流出去的原因，既保证召回率又避免误召回。

`SessionDB.search_messages()` 在拿到查询时按下面四条岔路分流。这套分流规则也是后面 小节 2.6 提到的 `recall_about` 在内部复用的同一套路由(FTS5 的索引面只有一份)，消费者从 hermes 内置工具到 SKU 自建工具都共用这一张表。

查询形态	走哪张表 / 改写	命中能力
CJK + libsimple 已加载（默认部署）	<code>messages_fts_trigram</code> <code>MATCH jieba_query(?)</code>	词级分词。“晚饭”、“妈妈”、拼音“mama”→妈妈、英文短词“yyds”全部命中
CJK ≥ 3 字, 无 libsimple	<code>messages_fts_trigram</code> <code>MATCH ' "<query>"'</code>	trigram 滑窗兜底；3-gram 切得出来即可命中
纯 ASCII / 英文 (≥ 2 字)	<code>messages_fts MATCH ?</code> (<code>unicode61</code>)	按 Unicode 词边界切分；布尔 / 短语 / 前缀 / 近邻全支持
其他	全 部 退 化 到 <code>content LIKE '%query%'</code>	无索引，全表扫描

表 2.3 FTS5 查询路由：按查询形态分流（`session_search` 与 `recall_about` 共用）

部署默认加载了 `wangfenjin/simple` 扩展 + `cppjieba` 字典，所以中文 查询走的是真正的“词级”分词路径（详见 小节 2.4 的对照表）——“妈妈”、“晚饭”、“yyds”、甚至“mama”→妈妈这种拼音回溯都能命中，比默认 trigram 的滑窗匹配召回率高一档。

2.4 Tokenizer 的选择与中文增强

`messages_fts_trigram` 这张表从 默认 `trigram` 切到 `simple+jieba` 是 hermes 内建的能力——不是改 schema 的破坏性升级，而是一个由扩展可用性 驱动 的自动开关。下面是切换前后的召回对照：

查询	trigram	simple+jieba
人工智能 (4 字)	✓	✓
晚饭 (2 字)	✗	✓
yyds (英文短词)	✗	✓
mama (拼音→中文)	✗	✓

表 2.4 trigram 与 simple+jieba 在中文短查询上的召回对比

❶ trigram 召不回 "晚饭"

trigram 的字面意思就是 **3 个字符的滑动窗口**——按 Unicode codepoint 计，不是 byte，所以 CJK 字符也是一字一格。它的工作方式是：

- **入索引时**：把消息文本切成所有连续的 3-字符片段。"我吃晚饭了" → "我吃晚" / "吃晚饭" / "晚饭了" 三条 token，倒排索引。
- **查询时**：FTS5 把查询当成 **substring search** 走 trigram 索引。SQLite 官方注记原话：“Substrings consisting of fewer than 3 unicode characters do not match any rows when used with a full-text query.” `MATCH '晚饭'` 仅 2 个字符，未达 trigram 下限 → 返回 0 行（无错误，静默 0）。

换到 jieba 分词器，按词典分词，索引里存的 token 是 "晚饭" 本身（不是 3-gram 碎片），查询 "晚饭" 也被分成同一个 token，索引 直接命中。

❷ 用户字典覆盖

专有名词 / 品牌 / 新词可以塞到 `$HERMES_HOME/jieba_user_dict.utf8` 这个 overlay 文件，下次 SessionDB 打开时会自动和 cppjieba 自带 baseline 合并、调 `jieba_dict()` 重载 —— 不需要改插件代码，也不需要触碰 vendor 目录里的字典源文件。

2.5 查询语法

查询	含义
小明 数学	包含两个词的消息（隐式 AND）
小明 OR 爸爸	布尔或
"考试 100"	完整短语匹配
小明 NEAR/3 数学	"小明"与"数学"距离 ≤ 3 个词
数学*	前缀匹配，命中"数学"/"数学课"/"数学题"
session_id:abcd*	按字段限定（fielded search）

表 2.5 FTS5 MATCH 子句的常用查询语法

底层调用如下：（`snippet()` 和 BM25 排名都是 FTS5 自带的，不需要额外计算。）

```
SELECT m.session_id, m.role,
       snippet(messages_fts_trigram, 0, '<<', '>>', '...', 16) AS hit,
       m.timestamp
FROM messages_fts_trigram t
JOIN messages m ON m.id = t.rowid
WHERE messages_fts_trigram MATCH jieba_query(?)
```

```
ORDER BY rank
LIMIT 20;
```

- x 已知限制

 - **FTS5 不支持语义近义词** —— “成绩”和“分数”是两个独立的索引项，需要语义召回时改用 MemoryProvider
 - **跨设备共享** —— 单文件 SQLite 是单机方案；多设备共享同一段记忆需要外挂 MemoryProvider（如 mem0 / supermemory / honcho）
 - **压缩与 simple+jieba 在底层兼容** —— 压缩不动 FTS schema，重写 messages 时索引随触发器自动更新；但压缩输出是中文摘要时，jieba 在新分词下的命中会跟原文不同

2.6 语音场景中的 FTS5

`session_search` 默认是 hermes CLI 上使用，返回带 BM25 分数、`>>>关键词<<<` 高亮记号、跨 session 拼接。这种输出不便于 TTS 朗读，`vox` 默认关闭了该能力。

`recall_about` 为了适配语音场景，返回简单列表方便朗读，同时按人召回。`topic` 过滤直接复用表 2.3 的同一路由，候选集预先用 `speaker_name = ?`（带索引）限制说话者。

i hermes 未显式导出路由规则
hermes 未显式导出路由规则，比如

```
SessionDB.build_topic_match_clause(topic: str) -> tuple[str, list]
```

让所有工具都靠它生成 SQL 片段，做到“路由规则一处定义、所有消费者 统一行为”。当下版本的 `recall_about` 直接借用了 hermes 内的几个 staticmethod helper（`_contains_cjk` / `_count_cjk`）。

2.6.1 recall_about 使用

使用方法是按指代调用、按场景克制：

合适调用的场景	典型会话
指代式回顾	“上次的事你还记得吗？”、“我那个”、“前段时间我跟你说的”
显式记忆查询	“你还记得我妈妈叫什么名字吗？”、“你还记得我说过最讨厌什么吗？”
跨日 / 跨 session 续聊	新一天的开场，孩子复提昨天的话题
多说话人歧义	家长问 “我儿子最近聊数学聊得多吗？”——agent 需要按 <code>speaker_name="小明"</code> 精准定位

表 2.6 `recall_about` 适用场景

应该跳过的场景	为什么
自由闲聊 / 情绪反应	当前轮没有指代、没有“记得”信号；调 = 浪费一次延迟预算
Session 第一轮	冷启动时索引里没有这个 speaker 的历史，零产出
Agent 自己讲故事 / 教学	此类回合的“事实依据”应该来自 persona / 工具，不应该靠召回
短话术应答（“嗯”、“好的”、“哈哈”）	TTS 已经处于“短回应”通道，再加调用会拉长无意义的等待

表 2.7 recall_about 不该被调用的场景

落实到工具描述（schema 的 description 字段）上，给模型的指令应该明确两件事：仅在指代式或显式记忆查询时调用 / 永远不要在回复中描述查询过程和元数据，仅将召回结果用自己的话融入对话。如果召回为空，agent 应正常说话，不要说“我没找到记录”这类话——空召回意味着“没有特别需要回想的内容”，正常对话即可。

3 Prompt Cache 与上下文压缩

3.1 大模型读写计费

主流 LLM 提供商都对相同前缀的请求做缓存：第二次请求如果与第一次共享 system prompt + tool schema + 历史前缀，提供商可以跳过对前缀的重新编码，价格降到大约 **十分之一**，延迟从首字节几百毫秒降到几十毫秒。一次 50 轮对话如果每轮都 miss 缓存，input token 成本是命中时的 10 倍，整个 session 的 LLM 账单可以到原本预算的 5-8 倍。

提供商	缓存命中价 (input)	未命中价
Anthropic	\$0.30 / 1M tokens	\$3.00 / 1M tokens
OpenAI	\$0.075 / 1M tokens	\$0.15 / 1M tokens
Gemini	部分模型支持，价格折扣 ~75%	否则全价

表 3.1 主流 LLM 提供商的 prompt cache 价格

① 缓存命中条件严格

缓存的命中条件很严格：从 system prompt 第一个 token 开始，到最近的 user 消息之前，必须 **逐字节相同**。任意一处变动（加一个 tool schema、把某条旧消息改一个标点、注入一段新的“用户偏好”）都会让前缀失配，提供商必须把整个前缀重新编码。

⚠ 容易破缓存的动作

- 中途切换 toolset（model “看到的工具集”变了）
- 重新加载 memory provider，把新的 memory 注入到 system prompt
- 修改历史消息的格式（哪怕只是把 [name] 加进 content）
- 旋转 system prompt（如把日期写进 prompt，每天变一次）
- 上下文压缩（详见 小节 3.3）

3.2 Cache 约束

为了把 cache hit 提升到 90% 以上，Hermes 将“哪些字节进入 cached prefix”严格分为三类。下图对应一次 API 入参的物理结构：

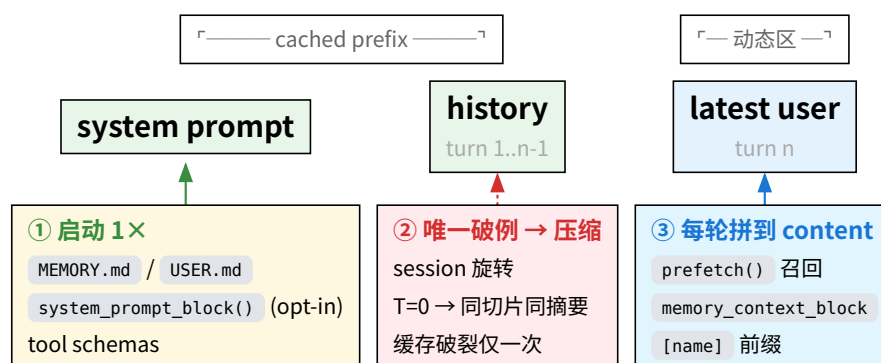


图 3.1 Prompt cache 三段三规则：① 启动一次性写入 → 进 system prompt；② 上下文压缩 → 唯一允许 in-place 改写历史；③ 所有 动态内容一律拼到 latest user message，避开 cached prefix。

备注

缓存命中率不是“做对了一点点”就有 90%，而是“任何一处做错就掉到 0%”。这是一个全有或全无（all-or-nothing）的指标。

3.3 上下文压缩

上下文压缩是唯一允许动历史的场景，且压缩后要求新的前缀稳定（同一段历史 在多次对话里压成同一个摘要）

使用一个独立的 LLM 调用 把中段摘要成几句话，保留头（最早的 system / 用户身份建立段）和尾（最近 N 轮原文），用摘要替换中间那段。

门槛	判定
① 主触发	<code>prompt_tokens >= threshold_percent × context_window</code> ，默认 <code>threshold_percent = 0.50</code> （可由 <code>ContextCompressor</code> 构造参数覆盖）
② 反颠簸	连续 2 次压缩 <code>savings < 10%</code> → 跳过本次（避免压无可压）

表 3.2 压缩触发门槛

压缩合法性检查：可压区间至少 `head + 3` 条消息（`head` 指 system prompt 加 `protect_first_n` 条前置消息，默认 3 条）。压缩流程独立于主对话循环 之外执行，避免阻塞面向用户的回合。

1. 选定 **压缩区间** (`messages[head:tail]`)，保留前 K 条与后 M 条原文
2. 调一个独立的 **压缩模型**（通常是更便宜的轻量模型）写摘要
3. **Session 旋转**：旧 session 收尾（标记 `end_reason='compression' + ended_at`）；新建一个延续 session，`parent_session_id` 指回旧 session；摘要作为一条 `role=system` 的 `messages` 行成为新 session 的开头
4. 整个过程被 `compression_locks` 表的 per-session 独占锁串行化。
5. 缓存破裂**仅一次**—— 之后这段历史的“新形态”再次进入 cache 路径

压缩是幂等的：T=0 + 输入确定 → 同切片同摘要

压缩是唯一允许动历史的场景，同一段历史在多次会话里应该压出同一个摘要。

1. **输入确定**—— 压缩的输入是确定的 `messages` 切片（`head:tail` 范围 加上 同一份会话历史），给定同一段历史拿到的 就是同一个 token 序列
2. **temperature = 0**—— 关掉模型的 sampling 随机性，每个位置 greedy 取 概率最高的 token

4 记忆能力扩展

关键字召回：FTS5，语义召回：MemoryProvider。

记忆主要分为两类：

1. **内置本地记忆**（MEMORY.md / USER.md）——零依赖、随仓库发布、写到磁盘的纯文本条目本（\$ 分隔的多行 entry，substring 匹配做增改删），最适合“事实条目 + 用户偏好”这种短文本
2. **MemoryProvider 抽象**——可插拔接口，把外部 SaaS（mem0 / supermemory / honcho）或自建向量库挂进来，覆盖“语义召回 / 跨设备同步 / 知识图谱”这类更重的需求

两层互不依赖：可以只开本地、只开 provider、两个都开、或两个都关。

4.1 内置本地记忆：MEMORY.md + USER.md

这是 Hermes 不依赖任何外部服务就能用的“记忆”——两份纯文本文件，会话启动时快照进 system prompt，agent 通过 memory 工具增删改条目。

文件	用途	容量上限
MEMORY.md	Agent 的工作笔记——环境事实、约定	2,200 字符 ≈ 800 token
USER.md	用户档案——身份、偏好、沟通风格、忌讳	1,375 字符 ≈ 500 token

表 4.1 本地记忆文件与字符上限

两份文件都存放在 \$HERMES_HOME/memories/，会话开始时一次性渲染进 system prompt 作为 frozen snapshot（参见小节 3）。Agent 中途调用 memory 工具写入磁盘后立即持久化，但要等到下一会话才会出现在 system prompt 中。

memory 工具采用 action 与 target 两个参数的组合：

参数	取值与含义
action	add / replace / remove —— 增 / 改 / 删一条条目
target	memory → MEMORY.md（默认）；user → USER.md

表 4.2 memory 工具的核心参数

① 本地记忆

- **有上限**：Hermes 为本地记忆设置的上限不大，但已足以容纳“环境信息 + 用户偏好”这一类最稳定的事实集合。容量将满时，agent 会自行合并或替换条目以腾出空间。
- **自动写入**：agent 会自动执行 add / replace / remove 三类写入动作。听到“我喜欢恐龙”时会主动落地，不需要用户明确说“请记住”。

4.2 多说话人本地记忆

MEMORY.md / USER.md 是 profile 级文件，每个 profile 仅有一份，路径中不携带 <user_id> / <speaker> 维度。然而语音设备的真实使用场景多为多名说话人共用一台设备：

- “我喜欢恐龙，特别是霸王龙”——用户 A（被监护人）说的
- “希望 A 早点睡觉别熬夜”——用户 B（监护人）说的

若 LLM 把这两条都视作同一个人的偏好，下次给 A 讲故事时就可能蹦出“早点睡觉别熬夜”这种本不属于 A 的语句，造成记忆泄露。

4.2.1 利用 LLM 纪律性分辨用户

依靠 LLM 自身的纪律：在 SOUL.md 加一段 `Multi-speaker memory discipline`，要求写入时给每条条目加 `[speaker_name]` 前缀，读取时按当前会话的 `user_name` 过滤——遇到 `[共享]` 始终命中，遇到其他前缀只有匹配该说话人时才采纳。

优势是零基础设施改动：profile 路径不变，hermes 主循环不需要感知多说话人；新增成员只需 gateway 把新名字写进 user message，纪律自动接管。代价是完全依赖模型的指令跟随能力。

实测验证下来，写入侧严格落地——所有条目都带 `[<speaker>]` 前缀，覆盖率 100%，跨会话再次披露同一事实时 agent 还会先扫描 USER.md 去重，不重复落档。但读取侧存在偏差：模型有时会把 `[A]` 标记的条目当成一般性偏好应用到所有说话人，这是同一份 system prompt 对所有说话人可见的天然边界——隔离的强弱完全取决于模型指令跟随能力，工程上不可控。要更强的保证，只能把过滤动作搬到模型之外。

4.2.2 Memory 预过滤

为消除读取侧对模型纪律的依赖，把过滤动作下沉到 hermes-core 的 prompt 拼装阶段。Hermes 内部的 `memory_tool` 按 `[<speaker>]` 与 `[共享]` 两种标签从 MEMORY.md / USER.md 条目里挑出“属于当前听众或共享”的部分，其他条目在物理上从该次 prompt 中移除。`system_prompt` 接收当前 session 的听众集合，在拼接 USER / MEMORY 段落前先调一次 filter。模型看到的 system prompt 已经是该听众视角下的子集，无从读取听众之外的偏好。

听众参数以 `"<name1>+<name2>+..."` 形式传入。Solo 场景退化为单名字字符串，与单说话人时代字节一致；group 场景下 filter 把字符串按 `+` 拆开并对任一名字匹配的条目放行，`[共享]` 条目无论何时都放行。

① 整体结论

- 写入侧：仍由 LLM 按 SOUL.md 纪律加前缀，覆盖率 100%。记录始终以当前发言者为 tag——group session 里 A 说话时仍写 `[A]`，不会写 `[A+B]`。
- 跨会话去重：agent 写入前先扫描 USER.md，已知事实不重复落档。
- 读取侧：由 hermes-core 在 prompt 拼装阶段按听众集合机械过滤，不再依赖模型纪律。

本章覆盖的是写入与读取的纪律——LLM 应当如何 tag、filter 应当如何按 tag 取舍。但听众集合从哪里来、单人和多人会话如何路由、外置记忆 provider 在多人场景下怎样协作，属于更基础的 session 模型问题。

4.3 MemoryProvider 抽象

`MemoryProvider` 是一个轻量抽象类，把高阶记忆能力做成可插拔后端。Hermes 主循环只跟它交互六个方法、按固定时机调用，不关心背后是 SaaS、向量库、知识图谱还是 LLM 摘要。

方法	调用时机	职责
<code>initialize</code>	会话启动时一次	建立连接、创建 schema、起后台线程
<code>system_prompt_block</code>	会话启动时一次	返回拼进 system prompt 的静态文本（自我介绍 / 状态行）；动态召回不走这里
<code>prefetch</code>	每轮 API 调用之前	返回这一轮要注入的召回上下文；必须低延迟（读后台缓存）
<code>queue_prefetch</code>	每轮结束之后	触发下一轮的后台召回，让下次 <code>prefetch</code> 直接命中缓存
<code>sync_turn</code>	每轮结束之后	把这轮对话写回后端；必须非阻塞（排队后台处理）
<code>shutdown</code>	会话关闭时	flush 队列、关连接

表 4.3 `MemoryProvider` 的生命周期方法

约定的工作流：实现里在 `queue_prefetch` 启一个后台线程做实际召回，把结果塞进进程缓存；下一轮 `prefetch` 直接读缓存，主循环零等待。

4.4 内置可选 backend

Hermes 仓库里收录了几个开箱即用的 provider，覆盖不同的取舍点。backend 实现差异很大，但对 Hermes 主循环来说接口完全相同。切换时只需在 `$HERMES_HOME/config.yaml` 里改一行 `memory.provider`。

backend	侧重点
<code>mem0</code>	全自动语义抽取，开发体验最好，云端 API
<code>supermemory</code>	跨设备同步 + 知识管理 UI
<code>honcho</code>	开源，自托管，per-user 长期偏好
<code>local</code>	纯本地嵌入向量库（chromadb），隐私 / 离线场景

表 4.4 `MemoryProvider`

5 插件系统

5.1 四条发现路径

Hermes 的插件系统是一个轻量的发现机制，运行时动态加载。

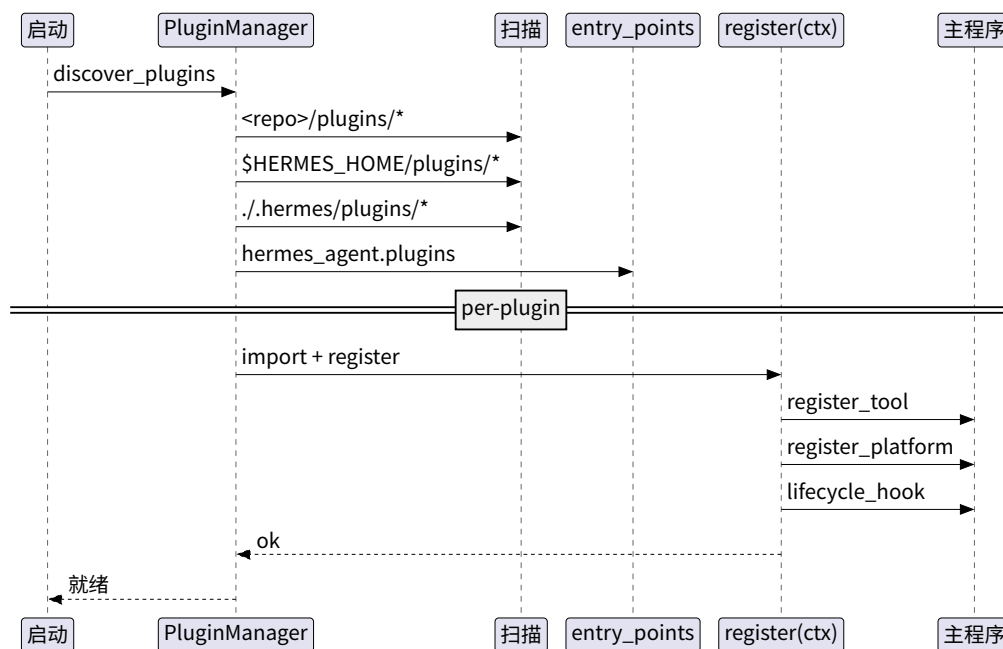


图 5.1 插件发现的四条路径 + register(ctx) 契约。后载入覆盖前者：项目级 > 用户级 > 仓库级 > entry_point。entry_point 零侵入主仓，是想跟随上游主线又不想 fork 时最干净的分发方式。

每个插件本质上是一个 Python 包，对外暴露一个 `register(ctx)` 函数。Hermes 提供的 `ctx` 是注册聚合器，提供 `register_tool` / `register_platform` / `register_lifecycle_hook` / `register_cli_command` 等方法。插件想做什么就往这个 `ctx` 上挂什么；Hermes 不限制插件能挂多少东西，只限制它怎么挂。

📎 entry point 的工作机制

第三方包声明 `[project.entry-points."hermes_agent.plugins"]` 映射到 `register(ctx)` 函数，hermes 启动时就会通过 `importlib.metadata.entry_points()` 自动发现并调用插件的 `register(ctx)`。

5.2 插件依赖拓扑

Vox 项目使用了 2 层 entry-point 模式：第一层将 vox 作为 hermes 插件被发现，第二层是 vox 自身开启一个新的 entry-point 组，供“产品 SKU”挂载。这使得同一硬件平台可承载多个产品形态，新增产品时既不需修改 hermes 主仓，也不需改动 vox 的平台层。

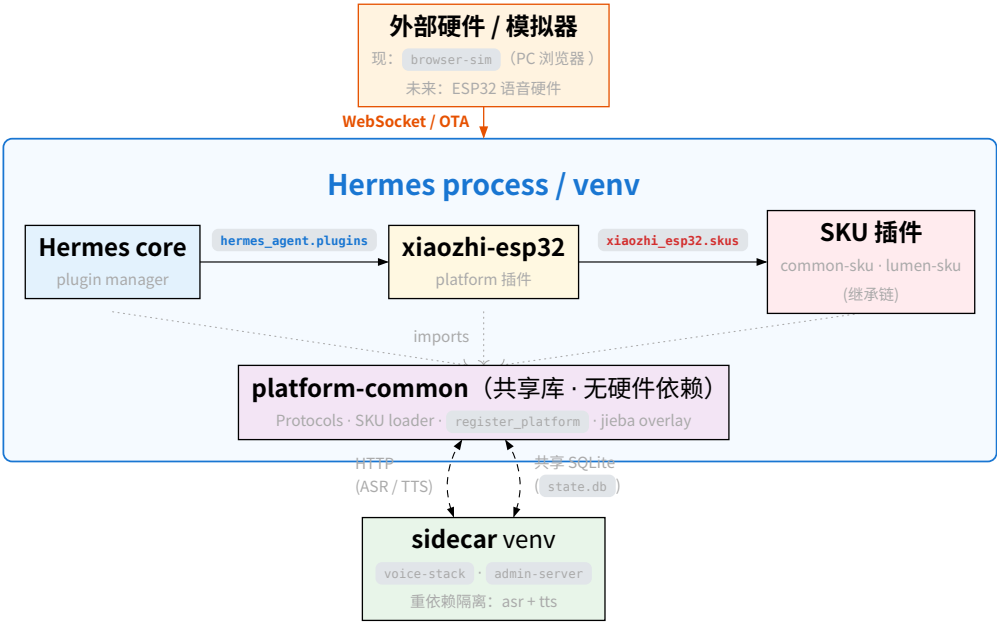


图 5.2 插件依赖拓扑：橙色“外部硬件 / 模拟器”通过 WebSocket / OTA 接入 蓝框内的 xiaozhi-esp32 适配器；现阶段是 browser-sim PC 模拟器，未来无缝替换为 ESP32 真机。蓝框内是同一个 Hermes venv，下方绿框是独立的 sidecar venv，跨 venv 走共享 SQLite + HTTP。

entry-point group	作用	声明者	消费者
hermes_agent.plugins	hermes 启动时发现所有主插件	xiaozhi-esp32	hermes
xiaozhi_esp32.skus	平台启动时发现所有 SKU 子插件	common-sku, lumen-sku	platform-common
[project.scripts]	Python 标准可执行脚本	sidecar	shell 环境

表 5.1 组件入口

- 设计原则
 - 重依赖隔离——保留在 sidecar venv，hermes 主进程保持轻量，启动快且与其他 hermes 部署互不干扰。
 - 硬件 / 产品解耦——硬件平台（xiaozhi-esp32）定义协议与 HAL，产品 SKU 定义 persona 与工具。
 - 跨 venv 接口——进程间不共享 Python 对象，仅共享 SQLite 与 HTTP 协议。

6 总结

Hermes 的路线选择可以浓缩为四点：

- 本地优先 —— SessionDB + FTS5，单文件 SQLite 满足绝大多数对话历史的存储与搜索需求，开箱即用，不依赖任何云服务
- 缓存命中 —— Prompt cache 决定一个对话产品的成本曲线，Hermes 把“不能破缓存”作为最高优先级的硬约束渗透到每一个修改点
- 分层可扩展 —— Memory provider 抽象 + 插件系统让更高阶的能力以正交方式加入：召回、跨设备同步、长期事实抽取都通过外挂实现，核心不为它们付出复杂度
- 工程隔离 —— plugin entry-point 让上层集成（语音设备）对 hermes 主仓零侵入

7 附录 A：对照参考 — Claude Code 的记忆与压缩

Claude Code 在 2025 下半年开源后，业界一次性看清了它对“记忆 + 压缩”的分层。Hermes 当前是“多面、平铺”的策略，Claude Code 是“严格分层”的策略。本节把它们的设计核心列出来，作为后续演进的参考标尺，并指出 Hermes 当前的对应实现与空白。两块内容（压缩流水线 + 记忆三级）一起读才能看出 Claude Code 整体的设计哲学。

7.1 压缩流水线：五层

Claude Code 把“context window 不够用”这件事拆成五层处理，每层代价不同、触发条件不同。

层	职责（解决什么问题）
预算 (budget)	按 model context window 估算“还能塞多少 token”。这一层不做修改，只回答“现在距离爆 context 还有多远”，给后面四层做触发依据。
剪枝 (prune)	清理低价值高占用的内容：失败的 tool 重试、大段重复的 stderr、被新结果覆盖的旧文件读取。剪枝在不调用 LLM 的前提下能省下惊人比例的 token，是廉价的第一道防线。
微压缩 (micro)	对单条仍然有价值但太啰嗦的消息做“段落级”压缩：一个 1000-token 的工具结果压成 200-token 的概要，但仍作为独立消息保留。微压缩保住“消息粒度”，只让每条变短，不打破消息序列结构。
折叠 (fold)	对一段“主题完整”的对话片段做更激进的合并：相邻的多条消息折叠成一条 system 角色的“事件摘要”。折叠改变了消息序列的形状，能省更多 token，代价是结构信息丢失更多。
自动压缩 (auto)	运行时根据预算、用户活动、空闲时间自动触发上面四层，无需用户介入。还包括反抖动逻辑（避免连续触发）和成本上限（一次压缩不能比省下的 token 还贵）。

表 7.1 Claude Code 压缩流水线的五层

这五层有清晰的“阶梯式代价”逻辑：预算（零成本）→ 剪枝（不调 LLM）→ 微压缩（cheap-model）→ 折叠（更贵的 LLM）→ 自动压缩（调度上面四层）。实际触发时按“从便宜到贵”逐层尝试：先剪枝，token 仍超就微压缩，再不够就折叠。这避免了“上来就重型摘要”这种暴力做法。

7.1.1 与 Hermes 4 阶段的对应

Claude Code 层	Hermes 对应 (agent/context_compressor.py)
预算	<code>should_compress()</code> + <code>threshold_tokens</code> + <code>_preflight_tokens</code> 三处估算
剪枝	<code>_prune_old_tool_results()</code> —— 把老的 tool result 替换成占位符 + 去重
微压缩	没有独立实现 。Hermes 直接进入摘要阶段，缺少“单条消息瘦身”这一中间步
折叠	<code>_generate_summary()</code> 把中段多条消息一次性摘成一条 system 消息，相当于折叠但没有“中间粒度”选项
自动压缩	<code>_compress_context</code> 在 <code>conversation_loop.py</code> 的 <code>preflight</code> 触发 + <code>max_compression_attempts=3</code> 错误重试

表 7.2 Claude Code 五层与 Hermes 实现的对应

完整算法在 `context_compressor.py` 里有一段官方注释：

Algorithm:
1. Prune old tool results (cheap pre-pass, no LLM call)

2. Protect head messages (system prompt + first exchange)
3. Find tail boundary by token budget
4. Summarize middle turns with structured LLM prompt
5. On re-compression, iteratively update the previous summary

读这段即可看出 Hermes 的 4 阶段（prune → 头尾保护 → 摘要 → 重组 + 迭代）实质上是 Claude Code 五层剪去头尾后的子集——缺的是“微压缩”这一层中间粒度。当某条 tool result 特别大（例如一次 web_extract 返回 30k token），它要么继续占用大量 token，要么被卷入 summary 中段而失去独立性。补上“微压缩”这一层并非大工程：在 `_prune_old_tool_results` 后增加一个 `_micro_compress_oversized_messages` 阶段，对单条 $\geq N$ token 的消息单独执行一次小模型摘要、保留为独立消息即可。

7.1.2 Hermes 反过来有什么 Claude Code 没明说的

1. **Session 旋转**——Hermes 在 `conversation_compression.py` 里把压缩当成一次“换 session”操作：旧 session 在 SessionDB 留底，新 session 从摘要开始；用户随时可以回去原 session 查证。Claude Code 公开资料里没强调这种持久化对照。
2. **压缩失败可中止**——`abort_on_summary_failure` 让用户可以选择“宁可报错也不要带着错的摘要继续”，避免静默的事实漂移。
3. **Plugin 替换**——`agent/context_engine.py` 把整个压缩器抽象为可替换接口，外部团队可基于自身需求采用更激进或更保守的策略替换默认实现，无需修改主仓。

7.2 记忆三级

Claude Code 的“记忆”是严格按生命周期分层的，每层职责清晰、互不混用。

层级	用途与生命周期
工作记忆	当前对话上下文里活着的部分：最近的 user / assistant 消息、tool 结果窗口、TODO 状态。一旦出窗就丢，模型靠“召回”再拿回来。
情节记忆	跨 turn 但仍属于同一 session 的事件序列：完整对话历史、压缩前的中段、上一次工具调用的 raw 输出。落到本地 store，可以倒带、可以二次检索。
语义记忆	跨 session 的“事实”：用户偏好、项目规范、长期约束。由专门的抽取器从对话里提炼，存到独立索引，与具体 session 解耦。

表 7.3 Claude Code 的三级记忆模型

7.2.1 三条核心原则

1. **指针式索引 (pointer-based)**——召回时返回的不是“摘要文本”，而是指向原始消息的**句柄** + 摘要 + 相关性分数。任何想要消费这段记忆的下游都能根据句柄回到原文，而不是被二手摘要锁死信息。
2. **非事实原则 (never invent facts)**——语义记忆抽取器被严格约束：“只报告对话里出现过的事实，不要做合理推断”。落到 prompt 上就是一段反复强调的硬指令 + 拒绝填空式输出。
3. **召回前必校验 (verify-before-recall)**——把一条记忆注入回上下文之前，对照原始消息做一次最小校验：摘要里的实体在原文是否真的出现过？时间戳是否合理？校验不过就丢弃这条召回。

7.2.2 Hermes 当前的落点

Hermes 的现状跟 Claude Code 三层有重叠也有空白：

三级	Claude Code	Hermes 现状
工作记忆	显式建模 + TODO	隐式（messages 列表 + 上下文压缩）
情节记忆	完整历史 + 句柄	SessionDB + FTS5 命中可回原文 ≈ 句柄
语义记忆	独立抽取器 + 索引	外挂 MemoryProvider, 目前都是纯 store / 召回式

表 7.4 Claude Code 三级记忆与 Hermes 现状对照

最大的空白是“语义记忆抽取器”——Hermes 把“记忆抽取”完全外包给 provider，但 provider 自己实现差异很大，大多数都不严格做“非事实原则”。这是后续可以单独建一个 Hermes 内置的 `facts_extractor` 来补的方向。